

Pole Placement via State Feedback

Putting the closed-loop poles exactly where you want them.

Ogata, *Modern Control Engineering*, Ch. 10.

In this tutorial you will:

- compute a feedback gain with Ackermann's formula (`acker`) and `place`,
- add a reference gain for zero steady-state error, and
- trade response speed against control effort by moving the poles.

Step through with **Ctrl+Enter**, or render a report with **publish**.

If (A, B) is completely controllable, full-state feedback $u = -Kx$ (plus a reference input r , $u = -Kx + r$) places the closed-loop poles – the eigenvalues of $A - BK$ – at **any** desired location in the complex plane (subject to complex poles occurring in conjugate pairs). The closed-loop characteristic equation is

$$|sI - (A - BK)| = 0$$

Contents

- Plant
- Before: The Open-Loop System Does Not Regulate
- Ackermann's Formula
- `place()` for Robust Multi-Pole Placement
- Closed-Loop Step Response with Reference Tracking
- What Changed: Poles Moved into the Left-Half Plane
- Effect of Desired Pole Location on Response Speed and Control Effort
- Try it yourself

Plant

$G(s) = \frac{1}{s(s+1)(s+2)}$ in controllable canonical form.

```
A = [0 1 0; 0 0 1; 0 -2 -3];
B = [0;0;1];
C = [1 0 0];
D = 0;
fprintf('Open-loop poles: '); disp(eig(A))
fprintf('Controllability rank = %d (n=%d)\n', rank(ctrb(A,B)), size(A,1))
```

```
Open-loop poles:      0      -1      -2
```

```
Controllability rank = 3 (n=3)
```

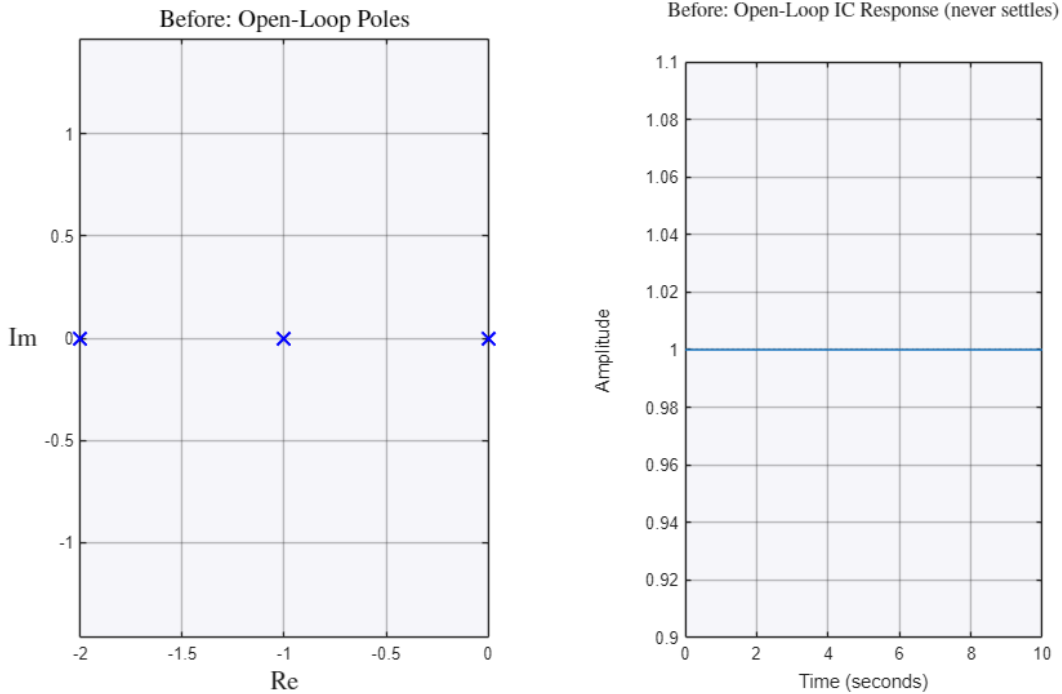
Before: The Open-Loop System Does Not Regulate

The open-loop A has a pole at the origin (an integrator) plus lightly damped modes, so from any disturbance the state never returns to zero. This is the "before" we are about to fix with state feedback.

```

figure
subplot(1,2,1)
plot(real(eig(A)),imag(eig(A)),'bx','MarkerSize',11,'LineWidth',1.5)
grid on; axis equal
title('Before: Open-Loop Poles','Interpreter','latex','FontSize',14)
xlabel('$\mathrm{Re}$','Interpreter','latex','FontSize',15)
ylabel('$\mathrm{Im}$','Interpreter','latex','FontSize',15)
set(get(gca,'YLabel'),'Rotation',0)
subplot(1,2,2)
initial(ss(A,B,C,D),[1;0;0],0:0.01:10)
title('Before: Open-Loop IC Response (never settles)','Interpreter','latex','FontSize',12)
grid on

```



Ackermann's Formula

For SISO systems, the feedback gain placing the poles at the roots of a desired characteristic polynomial $\alpha_c(s) = s^n + \alpha_1 s^{n-1} + \dots + \alpha_n$ is given directly by

$$K = [0 \ 0 \ \dots \ 1] \mathbf{C}^{-1} \alpha_c(A)$$

where $\mathbf{C}$ is the controllability matrix and $\alpha_c(A)$ is the matrix polynomial (Cayley-Hamilton-style substitution of A for s).

```

desired_poles = [-2+2j, -2-2j, -10];
K_acker = acker(A,B,desired_poles);
fprintf('\nAckermann gain K = '); disp(K_acker)
fprintf('Resulting closed-loop poles: '); disp(eig(A-B*K_acker))

```

```
Ackermann gain K =      80      46      11
```

```
Resulting closed-loop poles: -10.0000 + 0.0000i -2.0000 - 2.0000i -2.0000 + 2.0000i
```

Manual Ackermann computation (for verification): build $\alpha_c(s) = (s - p_1)(s - p_2)(s - p_3)$, then evaluate $\alpha_c(A)$.

```
alpha_c = poly(desired_poles); % desired char. polynomial coefficients
alpha_c_A = alpha_c(1)*A^3 + alpha_c(2)*A^2 + alpha_c(3)*A + alpha_c(4)*eye(3);
Cc = ctrb(A,B);
K_manual = [0 0 1]*(Cc\alpha_c_A);
fprintf('Manual Ackermann K = '); disp(K_manual)
```

```
Manual Ackermann K =      80      46      11
```

place() for Robust Multi-Pole Placement

‘place’ uses a numerically robust algorithm (Kautsky-Nichols-Van Dooren) and additionally supports multi-input systems; it requires distinct desired poles.

```
K_place = place(A,B,desired_poles);
fprintf('\nplace() gain K = '); disp(K_place)
```

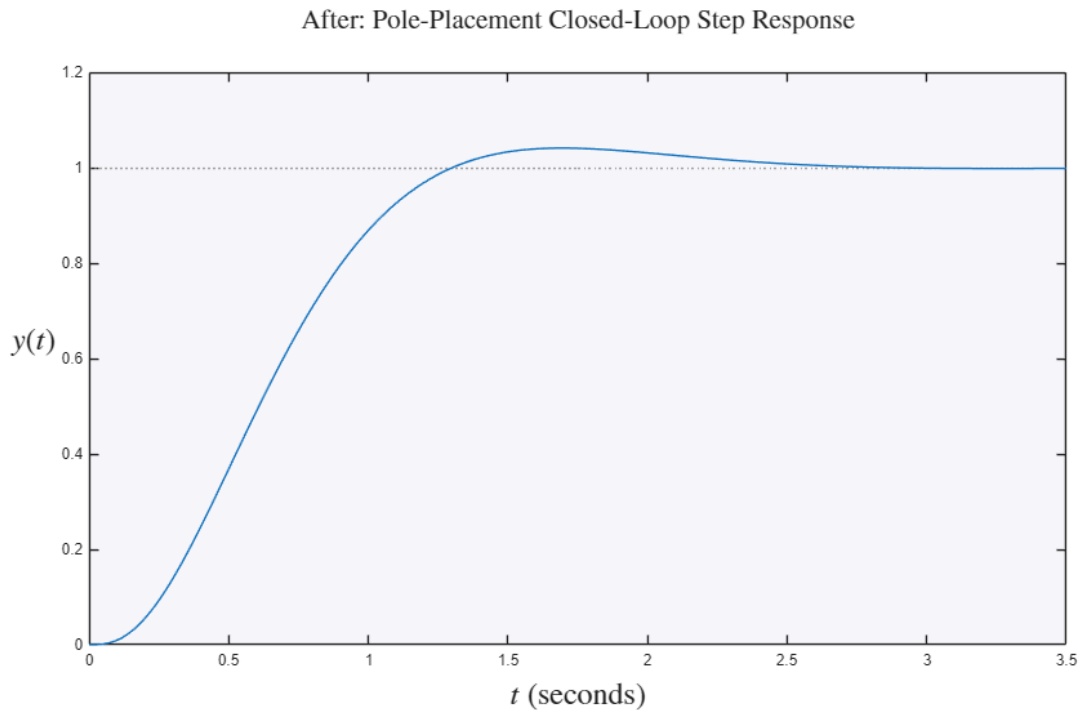
```
place() gain K =      80.0000      46.0000      11.0000
```

Closed-Loop Step Response with Reference Tracking

With $u = -Kx + N_r r$, choose N_r so the DC gain from r to y is 1 (zero steady-state error to a step reference), assuming $D = 0$:

$$N_r = \frac{-1}{C(A - BK)^{-1}B}$$

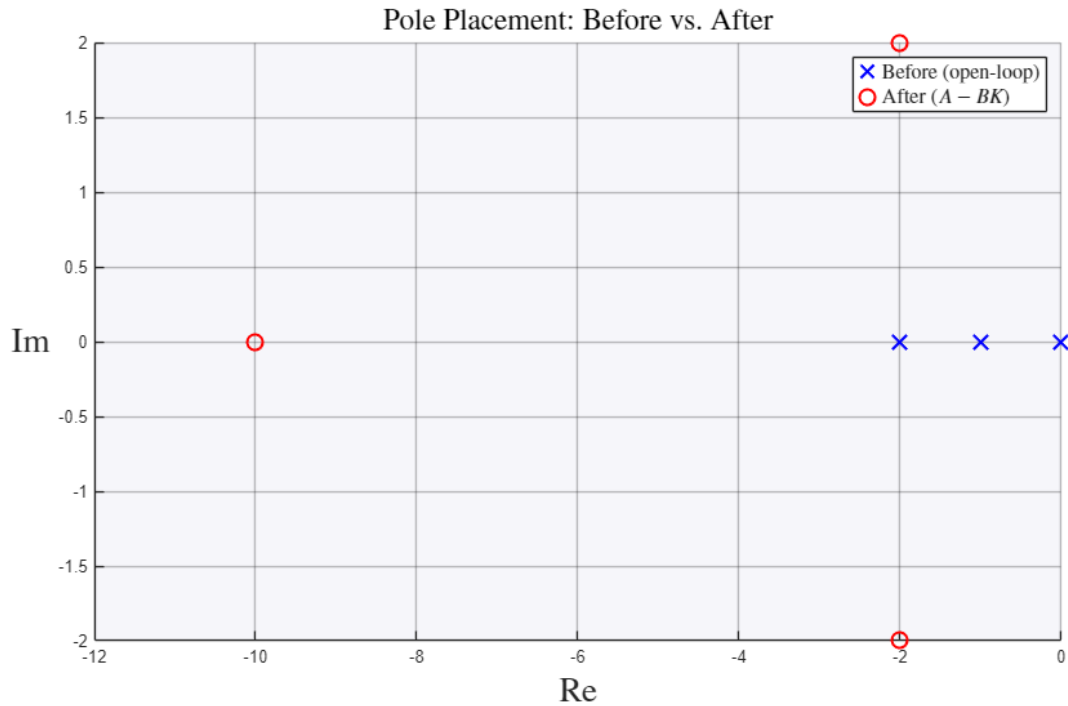
```
Nr = -1/(C*((A-B*K_acker)\B));
sys_cl = ss(A-B*K_acker, B*Nr, C, D);
figure
step(sys_cl)
title('After: Pole-Placement Closed-Loop Step Response','Interpreter','latex','FontSize',18)
ylabel('$y(t)$','Interpreter','latex','FontSize',20)
set(get(gca, 'YLabel'), 'Rotation', 0)
xlabel('$t$','Interpreter','latex','FontSize',20)
```



What Changed: Poles Moved into the Left-Half Plane

State feedback relocated every closed-loop pole from its open-loop position (one sat at the origin) to the chosen stable locations – this pole movement is exactly what produced the well-behaved step above.

```
figure
hold on
plot(real(eig(A)),imag(eig(A)),'bx','MarkerSize',12,'LineWidth',1.5)
plot(real(eig(A-B*K_acker)),imag(eig(A-B*K_acker)),'ro','MarkerSize',9,'LineWidth',1.5)
hold off
grid on
legend('Before (open-loop)','After ( $A-BK$ )','Interpreter','latex','FontSize',12)
title('Pole Placement: Before vs. After','Interpreter','latex','FontSize',17)
xlabel('Re','Interpreter','latex','FontSize',20)
ylabel('Im','Interpreter','latex','FontSize',20)
set(get(gca, 'YLabel'), 'Rotation', 0)
```



Effect of Desired Pole Location on Response Speed and Control Effort

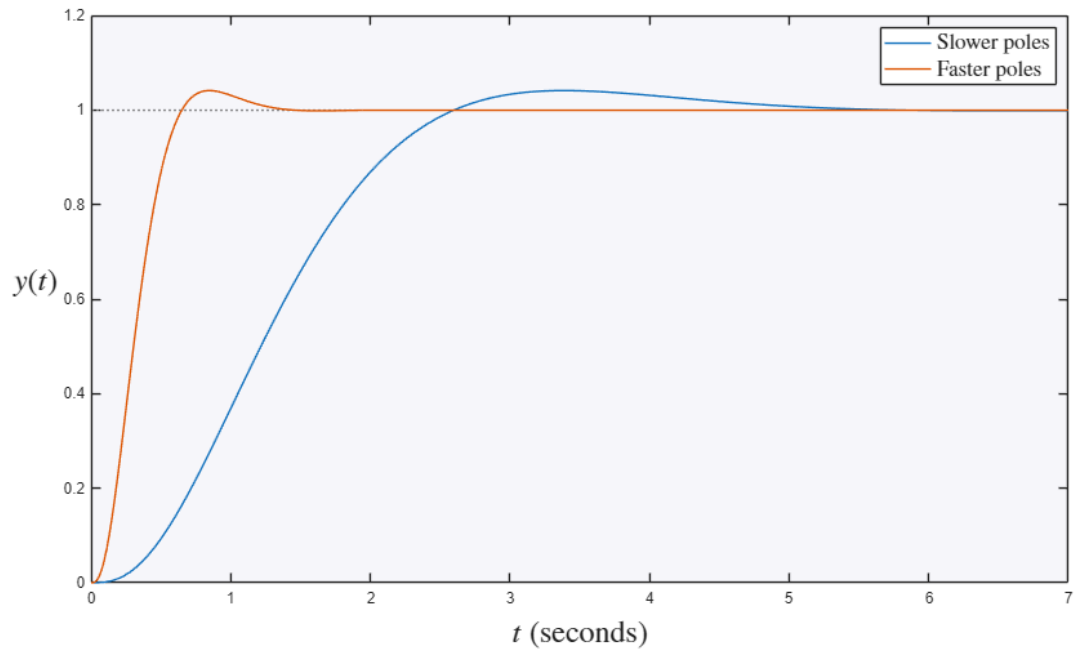
Faster (more negative) poles give a quicker response but typically larger gains K and larger initial control effort.

```
poles_slow = [-1+1j,-1-1j,-5];
poles_fast = [-4+4j,-4-4j,-20];
K_slow = acker(A,B,poles_slow);
K_fast = acker(A,B,poles_fast);
fprintf('\n||K_slow|| = %.2f, ||K_fast|| = %.2f\n', norm(K_slow), norm(K_fast))
```

```
Nr_slow = -1/(C*((A-B*K_slow)\B));
Nr_fast = -1/(C*((A-B*K_fast)\B));
sys_slow = ss(A-B*K_slow, B*Nr_slow, C, D);
sys_fast = ss(A-B*K_fast, B*Nr_fast, C, D);
figure
hold on
step(sys_slow)
step(sys_fast)
hold off
legend('Slower poles','Faster poles','Interpreter','latex','FontSize',14)
title('Pole Location vs. Response Speed','Interpreter','latex','FontSize',20)
ylabel('$y(t)$','Interpreter','latex','FontSize',20)
set(get(gca, 'YLabel'), 'Rotation', 0)
xlabel('$t$','Interpreter','latex','FontSize',20)
```

```
||K_slow|| = 14.70, ||K_fast|| = 668.08
```

Pole Location vs. Response Speed



Try it yourself

- Move the desired poles further left (faster) and notice `norm(K)` and the initial control effort grow.
- Set the reference gain to 1 (drop `Nr`) and watch a steady-state error appear.